

Medical Report Analyzer

1. Introduction

1.1 Project Overview

The Medical Report Analyzer is an AI-powered web application designed to bridge the gap between complex medical documentation and patient comprehension. By leveraging large language models, the system translates clinical terminology, lab values, diagnoses, and prescriptions into plain, readable language that any patient can understand without a medical background.

1.2. Problem Statement

Medical reports are filled with clinical terminology, abbreviations, and technical jargon that is difficult for the average person to understand. Patients receiving blood test results, discharge summaries, lab reports, or prescriptions are often left confused about what the findings mean and what actions they should take.

Why the problem matters:

- Millions of patients receive medical reports each year without the ability to understand them fully.
- Misunderstanding a report can lead to delayed treatment, unnecessary anxiety, or poor health decisions.
- Patients often rely on costly follow-up appointments just to have a doctor explain what the report says.

Why existing solutions are insufficient:

- Generic search engines return broad medical information, not personalized analysis of a specific report.
- Rule-based tools cannot handle the variability in report formats, abbreviations, and layouts across hospitals and labs.
- No current mainstream tool offers a conversational, interactive way to ask questions about your own report.

Why AI is relevant:

- Large language models can read unstructured medical text, understand clinical context, and rephrase findings in plain language.
- Conversational AI enables a natural Q&A experience that static tools simply cannot replicate.

1.3 Objectives

- Provide patients with instant, plain-language summaries of their medical reports.
- Extract and highlight key findings such as diagnoses, lab values, and abnormal results.
- Offer an interactive chatbot for follow-up questions grounded in the uploaded report.
- Securely store reports, summaries, and chat histories per authenticated user account.
- Support both text-pasted and image/scanned report uploads via OpenAI Vision.

1.4 Scope

The system is scoped to serve individual patients who receive medical documents from hospitals, clinics, or diagnostic laboratories. The application will:

- Accept medical reports as typed text, PDF files, or scanned images.
- Generate AI-powered summaries and structured key-finding extractions.
- Provide a conversational chatbot interface to answer report-specific questions.
- Maintain a per-user report history accessible at any time.

Out of scope: direct integration with hospital EHR systems, real-time diagnostic advice, and multi-language support in version 1.0.

2. Functional Requirements

2.1 Authentication & User Management

Req. ID	FR-01
Name	User Registration
Description	New users can create an account by providing a name, email address, and password. The system validates input and stores credentials securely.

Actors	Guest User
Pre-condition	User is not registered.
Post-condition	Account created; user is redirected to the login page.

Req. ID	FR-02
Name	User Login
Description	Registered users can log in using their email and password. On success, the system issues a JWT token for session management.
Actors	Registered User
Pre-condition	Valid account exists.
Post-condition	JWT issued; user is redirected to the dashboard.

Req. ID	FR-03
Name	User Logout
Description	Authenticated users can log out, invalidating their session token.
Actors	Authenticated User
Pre-condition	User is logged in.
Post-condition	Session token invalidated; user redirected to login page.

2.2 Report Input & Upload

Req. ID	FR-04
Name	Text-Based Report Input
Description	Users can paste the content of a medical report directly into a text input field for analysis.
Actors	Authenticated User
Pre-condition	User is logged in.

Post-condition	Report text is stored and sent for AI analysis.
-----------------------	---

Req. ID	FR-05
Name	PDF File Upload
Description	Users can upload a PDF file containing their medical report. The system extracts the text content for analysis.
Actors	Authenticated User
Pre-condition	User is logged in; valid PDF file available.
Post-condition	PDF is stored in Supabase Storage; text extracted and sent for analysis.

Req. ID	FR-06
Name	Image / Scanned Report Upload
Description	Users can upload a scanned or photographed image of a report. OpenAI Vision processes the image to extract text.
Actors	Authenticated User
Pre-condition	User is logged in; image file (JPG/PNG) available.
Post-condition	Image stored; OCR performed; extracted text sent for AI analysis.

2.3 AI-Powered Analysis

Req. ID	FR-07
Name	Plain-Language Summary Generation
Description	The system sends the report content to OpenAI and returns a structured, plain-language summary understandable by non-medical users.
Actors	Authenticated User, AI Service
Pre-condition	Report content is available.
Post-condition	Summary displayed on the analysis dashboard within seconds.

Req.ID	FR-08
Name	Key Findings Extraction

Description	The AI extracts and categorizes key findings including diagnoses, medications, lab values, and abnormal results.
Actors	Authenticated User, AI Service
Pre-condition	Report content is available.
Post-condition	Structured key findings displayed with abnormal values flagged in red.

2.4 Interactive Q&A Chatbot

Req. ID	FR-09
Name	Chatbot Question & Answer
Description	Users can ask natural-language questions about their report. The chatbot responds based on the specific report content.
Actors	Authenticated User
Pre-condition	A report has been analyzed and is in context.
Post-condition	AI-generated answer displayed in the chat interface.

Req. ID	FR-10
Name	Conversation History Persistence
Description	All chatbot messages for a given report session are saved so users can review previous questions and answers.
Actors	Authenticated User
Pre-condition	User has interacted with the chatbot.
Post-condition	Chat history saved and visible on revisiting the report.

2.5 Report History

Req. ID	FR-11
Name	View Past Reports
Description	Users can view a list of all previously submitted reports along with their AI summaries.
Actors	Authenticated User

Pre-condition	At least one report has been submitted.
Post-condition	List of past reports displayed with timestamps and summary previews.

Req. ID	FR-12
Name	Resume Past Report Chat
Description	Users can select a past report and continue asking questions via the chatbot.
Actors	Authenticated User
Pre-condition	A past report with chat history exists.
Post-condition	Report context reloaded; chatbot resumes with full history.

3. Non-Functional Requirements

3.1 Performance

- AI-generated summaries must be returned within 10 seconds for reports under 5,000 words.
- API response time for authentication endpoints must be under 500 ms under normal load.
- The frontend dashboard must load within 3 seconds on a standard broadband connection.

3.2 Security

- All user passwords must be hashed using bcrypt with a minimum cost factor of 10.
- JWT tokens must expire after 24 hours and be verified on every protected API request.
- Uploaded files must be stored in private Supabase Storage buckets accessible only by the owning user.
- All data in transit must be encrypted using HTTPS/TLS 1.2 or higher.

3.3 Usability

- The UI must be fully responsive, supporting desktop, tablet, and mobile screen sizes.
- Plain-language summaries must avoid medical jargon and target a reading level of Grade 8 or below.

- Abnormal lab results must be visually distinguished (e.g., highlighted in red) for immediate user attention.

3.4 Reliability & Availability

- The application must target 99.5% uptime, leveraging Vercel and Render managed hosting.
- In the event of an AI service timeout, the user must receive a clear error message with a retry option.

3.5 Scalability

- The backend must handle at least 100 concurrent users without performance degradation.
- Database queries must use indexed columns on `user_id` and `report_id` fields for efficient lookup.

3.6 Maintainability

- The codebase must follow modular architecture separating frontend, backend, and AI service layers.
- All API endpoints must be documented using OpenAPI / Swagger specification.

User Roles

The system defines distinct user roles with specific permissions and access levels.

Patient / End User

The primary user of the system. Patients are individuals who receive medical reports and want to understand them better.

- Register and manage their personal account.
- Upload, view, and delete their own medical reports.
- Receive AI-generated plain-language summaries.
- Interact with the chatbot to ask questions about their reports.
- View and search their complete report history.

- Export or download report summaries.
- Cannot access other users' data or system administration functions.

System Administrator

System administrators oversee platform operations, user management, and system health monitoring.

- Access the admin dashboard with system-wide analytics.
- View aggregated usage statistics (no access to individual medical content).
- Manage user accounts (suspend, reactivate, delete).
- Monitor system performance and error logs.
- Configure system parameters and AI model settings.
- Cannot view the content of users' medical reports or chat histories (privacy protection).

Role Permissions Matrix

Feature	Patient	Admin
Register/Login	✓	✓
Upload Report	✓	✗
View AI Summary	✓	✗
Use Chatbot	✓	✗
View Report History	✓ (Own only)	✗
Delete Report	✓ (Own only)	✗
Export Summary	✓	✗
System Analytics	✗	✓
User Management	✗	✓

View Medical Content	✓ (Own only)	✗ (Privacy)
----------------------	--------------	-------------

Use Cases

This section describes the interactions between users (actors) and the Medical Report Analyzer system.

Use Case Diagram Description

The Use Case Diagram illustrates the primary interactions between the two actors (**Patient** and **System Admin**) and the core system functionalities. The diagram shows :

- **Patient Actor:** Interacts with all core features including authentication, report upload, AI analysis, chatbot, and history management.
- **System Admin Actor:** Interacts with system monitoring and user management features via the admin dashboard.
- **Core System:** Encompasses all functional modules: Auth Module, Report Module, AI Analysis Module, Chatbot Module, and History Module.
- **External Services:** OpenAI API (for summarization, extraction, chatbot, vision/OCR), Supabase (database & storage), and Email Service.

Detailed Use Cases

UC-001: User Registration

Actor: Patient

Precondition: User has access to the internet and the application URL.

Main Flow:

1. User navigates to the registration page.
2. User enters email, password, and confirms password.

3. System validates input format and uniqueness of email.
4. System creates user account and sends verification email.
5. User verifies email by clicking the link.
6. System activates the account and redirects to login.

Postcondition: User account is created and verified. User can now log in.

Alternative Flows: If email already exists, system displays error. If password doesn't meet complexity, system shows requirements.

UC-002: User Login

Actor: Patient

Precondition: User has a registered and verified account.

Main Flow:

1. User navigates to the login page.
2. User enters email and password.
3. System validates credentials.
4. System generates JWT token and establishes session.
5. System redirects user to the dashboard.

Postcondition: User is authenticated and can access protected features.

Alternative Flows: If credentials are invalid, system displays error message without revealing which field is incorrect.

UC-003: Upload Medical Report

Actor: Patient

Precondition: User is logged in.

Main Flow:

1. User navigates to the upload page.
2. User selects upload method: text input or file upload.
3. If file upload: user selects PDF or image file.
4. System validates file type and size.
5. System stores the file in Supabase Storage.
6. System triggers AI analysis pipeline.
7. System displays processing status to user.

Postcondition: Report is uploaded, stored, and queued for AI analysis.

Alternative Flows: If file type is unsupported or exceeds size limit, system rejects upload with clear error message.

UC-004: View AI Summary

Actor: Patient

Precondition: User has uploaded a report and AI analysis is complete.

Main Flow:

1. System receives AI-generated summary from OpenAI API.
2. System structures the summary into sections: overview, key findings, abnormal results.
3. System displays the summary on the dashboard with visual indicators.
4. User reads the plain-language summary.
5. User can expand sections for more detail.

Postcondition: User understands their medical report in plain language.

Alternative Flows: If AI analysis fails, system displays error and allows retry.

UC-005: Ask Follow-up Questions (Chatbot)

Actor: Patient

Precondition: User has a completed AI summary for their report.

Main Flow:

1. User opens the chatbot panel from the summary dashboard.
2. User types a natural language question about their report.
3. System sends the question along with report context to OpenAI API.
4. System receives and displays the AI-generated response.
5. User can ask follow-up questions in the same session.
6. System stores the conversation in chat history.

Postcondition: User receives personalized answers to their medical report questions.

Alternative Flows: If the question is unrelated to the report, chatbot politely redirects to report-specific queries.

UC-006: View Report History

Actor: Patient

Precondition: User is logged in and has previously uploaded reports.

Main Flow:

1. User navigates to the 'My Reports' or History page.
2. System queries the database for all reports belonging to the user.
3. System displays a list of reports with metadata.
4. User selects a specific report to view.
5. System loads the full summary and chat history for that report.

Postcondition: User can review any past report and its associated analysis.

Alternative Flows: If no reports exist, system displays an empty state with a prompt to upload.

UC-007: Delete Report

Actor: Patient

Precondition: User is logged in and has at least one uploaded report.

Main Flow:

1. User navigates to the report history page.
2. User selects a report and clicks 'Delete'.
3. System displays a confirmation dialog.
4. User confirms deletion.
5. System deletes the report file from storage, summary from database, and associated chat history.
6. System updates the report list.

Postcondition: Report and all associated data are permanently removed.

Alternative Flows: User can cancel the deletion before confirmation.

UC-008: Monitor System (Admin)

Actor: System Administrator

Precondition: Admin is logged in with admin credentials.

Main Flow:

1. Admin navigates to the admin dashboard.
2. System displays aggregated analytics: total users, reports processed, average processing time.

- 3.Admin views user management interface.
- 4.Admin can suspend or delete user accounts if necessary.

Postcondition: Admin has visibility into system health and user activity.

Alternative Flows: Admin cannot view the content of individual medical reports due to privacy constraints.

Data Requirements

Database Schema Overview

The system uses **PostgreSQL** (via Supabase) as the primary relational database. The following tables are defined:

Table	Description	Key Fields
users	Stores registered user accounts and authentication data.	id (PK), email, password_hash, full_name, created_at, is_verified, role
reports	Stores metadata about uploaded medical reports.	id (PK), user_id (FK), file_name, file_url, report_type, upload_date, status
summaries	Stores AI-generated plain-language summaries.	id (PK), report_id (FK), summary_text, key_findings (JSON), abnormal_flags (JSON), generated_at
chat_history	Stores conversation threads between user and chatbot.	id (PK), report_id (FK), user_id (FK), message, response, timestamp, session_id

sessions	Tracks active user sessions for JWT management.	id (PK), user_id (FK), token, refresh_token, created_at, expires_at
----------	---	---

Data Flow Description

Upload Flow: User → Frontend → Express API → Supabase Storage (file) + PostgreSQL (metadata) →

OpenAI API (analysis) → PostgreSQL (summary) → Frontend (display)

Chat Flow: User Question → Frontend → Express API → PostgreSQL (fetch report context) → OpenAI

API (with context) → PostgreSQL (store Q&A;) → Frontend (display response)

History Flow: User Request → Frontend → Express API → PostgreSQL (query user's reports) →

Frontend (display list)

Interface Requirements

User Interfaces

UI-001: Landing Page: Clean, welcoming page with project description, features overview, and call-to-action buttons (Register/Login).

UI-002: Authentication Pages: Registration form, login form, password reset form with validation feedback.

UI-003: Upload Dashboard: Central interface with text input area and drag-and-drop file upload zone with format indicators.

UI-004: Analysis Dashboard: Displays AI summary in structured cards: Overview, Key Findings (with color-coded abnormal flags), Medications, and Recommendations.

UI-005: Chatbot Panel: Slide-out or embedded chat interface showing conversation history, message input, and typing indicators.

UI-006: Report History Page: Grid or list view of past reports with search, filter, sort, and pagination.

UI-007: Settings/Profile Page: User account management, password change, and data export options.

UI-008: Admin Dashboard: Analytics cards, user management table, and system health indicators (admin-only).

Software Interfaces

SI-001: OpenAI API: RESTful API integration for text summarization, entity extraction, chatbot responses, and vision/OCR processing.

SI-002: Supabase PostgreSQL: Database connection via Supabase client library for CRUD operations.

SI-003: Supabase Storage: Object storage API for file upload, retrieval, and deletion.

SI-004: Email Service: Integration with email provider (e.g., SendGrid, Resend) for verification and password reset emails.

SI-005: Frontend-Backend: REST API communication via Axios/fetch with JSON payloads and JWT in Authorization header.

Hardware Interfaces

The system is a web-based application with no direct hardware interfaces. Users interact through standard computing devices (desktop computers, laptops, tablets, smartphones) with internet connectivity, web browsers, and input devices (keyboard, mouse, touchscreen). File upload requires devices with camera or file system access for image/PDF selection.

Constraints

- Regulatory:** The application provides informational summaries only and must not be marketed as a medical diagnosis or treatment tool. HIPAA compliance is required for handling PHI.
- Technical:** AI analysis depends on OpenAI API availability and rate limits. The system must handle API downtime gracefully.
- Budget:** Development and deployment costs must remain within the project budget, utilizing free tiers where possible (Vercel Hobby, Render Free, Supabase Free).
- Time:** The project must be completed within the academic semester timeline.
- Language:** Initial release supports English-language reports only.